

Capítulo 5: El plano de control en la capa de red

El objetivo de los algoritmos de ruteo es encontrar el camino menos costoso en la capa de red. Hay dos mecanismos para instalar y mantener las tablas de forwarding y flujo en los routers:

1. Algoritmos **por router**: cada router tiene un componente que se comunica con otros para computar los valores de su tabla.
2. Algoritmos **lógicamente centralizados**: un controlador centralizado computa las tablas y las instala en los routers.

Los algoritmos se construyen sobre la idea de que un grupo de routers conforma un grafo, donde cada nodo es un router, cada arista un enlace, y cada peso representa el costo del enlace (monetario, en tiempo por throughput, longitud, etc).

Los algoritmos de ruteo se pueden clasificar de varias formas.

1. **Por conocimiento de la red**:
 - a. **Centralizados**: se conoce la red completa, también llamados **link-state**.
 - b. **Descentralizados**: el cómputo del camino se hace de forma iterativa, ya que el router solo conoce a sus vecinos, con quienes va intercambiando información hasta saber lo suficiente como para construir el camino.
2. **Por volatilidad de cambio de los caminos**:
 - a. **Estáticos**: las rutas cambian muy lentamente, generalmente por la intervención humana.
 - b. **Dinámicos**: las rutas cambian por la variación del tráfico en la red y el cambio de topología (por ejemplo, se corta un enlace o aparece otro).
3. **Por sensibilidad al tráfico en la red**:
 - a. **Load-sensitive**: el costo de un enlace puede depender del tráfico en la red.
 - b. **Load-insensitive**.

Algoritmos Link-State

Se conoce toda la red: todos los routers, enlaces y costes de enlaces. Esto se logra al hacer que todos los routers **broadcasteen** mensajes de **link-state** a todos los demás nodos de la red. Hay dos algoritmos.

1. **Dijkstra**. Es un algoritmo iterativo; para la iteración k , se conocen los caminos menos costosos hasta otros k nodos.
2. **Prim**.

Los algoritmos son $\mathcal{O}(n^2)$ respecto a la cantidad de routers.

Dijkstra

Tenemos un nodo que pertenece a un grafo, y tiene determinados vecinos. Dijkstra calcula la ruta de menor coste desde el nodo hasta todos los demás nodos del grafo, de ahí su complejidad temporal.

Se parte de una lista de nodos por visitar (solo con el nodo de origen), una lista de nodos visitados (vacía), y una lista de distancias hacia cada otro nodo, inicializada con distancias en el infinito para cada nodo distinto del nodo de origen.

Se agrega el nodo de origen a la lista de visitados, y se lo elimina de la lista de nodos por visitar. Por cada nodo revisamos sus vecinos y vemos si el costo de viajar a él + el costo de viajar desde él hasta ese vecino es menor o mayor a la distancia que ya teníamos guardada; cuando terminamos de ver sus vecinos, los agregamos a la lista de nodos por visitar, y se repite iterativamente.

Algoritmos Distance-Vector

Características:

1. Es **distribuido**: cada router solo habla con sus **vecinos directos**.
2. Es **iterativo**: el algoritmo continúa solo hasta que no haya más comunicación entre vecinos.
3. Es **asíncrono**: los routers no deben lockearse entre ellos.

Ruteo en sistemas autónomos

Un **sistema autónomo (AS)** es un grupo de routers gestionados por el mismo control administrativo. Generalmente se da cuando ISPs quieren gestionar sus

propios routers.

Los routers de un **AS** utilizan el mismo algoritmo de ruteo y tienen información del resto de routers. El algoritmo de ruteo que ejecutan se denomina ***intra-anonymous system (intra-AS) routing protocol***.

Routing Information Protocol (RIP)

Es un protocolo de enrutamiento de tipo **distance-vector** que aplica **Bellman-Ford**. Cada nodo se va **compartiendo su tabla de vecinos a sus vecinos**, y cuando la reciben **la usan para actualizar la que tienen**. La estrategia de RIP va por saltos; tomamos la ruta con menos saltos, que es lo que representa el costo de cada arista (1).

Cuando llega una tabla de un vecino directo, por cada vecino anunciado en esa tabla tengo que actualizar la mía; calculo la cantidad de saltos hasta llegar al vecino que anunció su tabla, y le sumo 1 a cada camino (porque tengo que ir al vecino + tengo que ir al vecino del vecino), y reviso si eso es menos que lo que ya me tomaba antes del anuncio.

Problemas conocidos: Count-To-Infinity, cuando una distancia tiende a infinito en la tabla de ruteo.

Sucede cuando los nodos tienen información vieja de otros nodos que ya no están en el sistema. Cuando un nodo X vecino de Y se va del grafo, Y ya no anuncia a X como vecino, pero los demás nodos sí pueden hacerlo, haciéndole pensar que puede llegar a X a través de ellos; como cada distancia se va aumentando de a 1 porque contamos saltos, la distancia hasta X tiende a infinito, de ahí el nombre del problema.

Open Shortest Path First (OSPF)

Es un protocolo de enrutamiento de tipo **link-state**, aplica un ***flooding*** de mensajes de **link-state** (inunda el SA de mensajes), y usa **Dijkstra**.

Ventajas:

1. Seguridad. Se puede implementar autenticación en OSPF.
2. Pueden coexistir múltiples caminos con el mismo costo y todos ser usables.
3. Soporte integrado para ruteo unicast y multicast.
4. Permite jerarquización.

Ruteo entre SA: BGP

Border Gateway Protocol (BGP), es el protocolo que se utiliza para rutear entre **SA**s. Es un *exterior gateway protocol*.

Un router puede ser **interno (internal)**, si solo habla con routers dentro del mismo **SA**, o **de frontera (gateway)**, si habla con routers de otros **SA**.

Cada **SA** tiene un **prefijo**, que es asignado cuando se anuncia (**BGP** hace que todos los otros **SA** lo conozcan), y **BGP** rutea para la mejor ruta sobre prefijos.

BGP se monta sobre **TCP (puerto 179)** con conexiones temporales. Se dice que son *conexiones BGP*. Pueden ser entre routers del mismo SA (internas, iBGP) o entre routers de distintos SA (externas, eBGP).

Cuando un router avisa de un prefijo, incluye varios atributos BGP (a eso se le dice **ruta**). Los más importantes son:

1. **AS-PATH**: la lista de prefijos de SA por los que el aviso pasó. Los prefijos se van agregando al inicio (tipo queue).
2. **NEXT-HOP**: la IP del router al que debo enviar los paquetes.

El **NEXT-HOP** varía si estamos en eBGP o iBGP. En eBGP, NEXT-HOP suele ser la IP del router de frontera que me envió la ruta (o de algún otro router del SA que me la envió), porque el SA debe aprender que los paquetes que quieran ir a ese otro SA deben ir por ese router. En iBGP, los routers internos no conocen los del otro SA, por eso suele hacerse que NEXT-HOP sea realmente otro router dentro del mismo SA, que suele ser la IP del router de frontera que recibió la ruta.

Algoritmos de ruteo BGP

El más simple es el **hot potato routing**, donde el camino elegido es aquel de menor costo al router NEXT-HOP. Es *greedy*, porque la idea es reducir el coste dentro del AS, asumiendo que eso reduce el coste global del anuncio de la ruta.

El que se usa es el **route-selection**, en el que el input del algoritmo es un conjunto de todos los caminos hacia el prefijo dado, que fueron aprendidos y aceptados por el router. Puede suceder que haya múltiples caminos para un mismo prefijo, en cuyo caso hay reglas de eliminación (en orden descendente por caso de uso):

1. Los caminos tienen un *local preference*, y se eligen los caminos con mayor valor.
2. Si quedan más, se toma el AS-PATH más corto.

3. Si quedan más, se aplica hot potato.
4. En última instancia, se utilizan identificadores BGP.

IP Anycast

Se puede utilizar BGP para implementar IP anycast. Durante la etapa de configuración, si se tienen varios servidores, se les asigna la misma dirección IP, y se anuncia esa misma. Los routers BGP interpretan eso como diferentes rutas a la misma dirección física, y las tablas de ruteo quedan configuradas para elegir la mejor ruta hacia esa dirección.

El plano de control SDN

Las SDN se dividen en dos componentes: el controlador y las aplicaciones de control de red.

Las funciones del controlador se pueden dividir en capas:

1. **Capa de comunicación:** la interfaz de frontera sur/southbound para comunicarse con el PD.
2. **Capa de gestión de estado de la red:** el controlador debe tomar decisiones en función del estado de toda la red, y guardar copias de las tablas de flujo de los switches.
3. **Capa de interfaz para las aplicaciones de control de red:** la interfaz de frontera norte/northbound para que puedan implementarse aplicaciones sobre el controlador. Permiten leer y escribir el estado de la red y las tablas de flujo.

ICMP

Protocolo utilizado para enviar información de la capa de red, típicamente para informar errores. **Viajan dentro de un datagrama IP**, tienen un **type** y un **code**, contienen el header y los primeros 8 bytes del datagrama IP que causó que se genere el mensaje ICMP.

Gestión de redes y SNMP

Componentes clave de la gestión de red:

1. El **managing server** es la aplicación que controla la colección, el procesamiento, el análisis, y/o el display de la información.

2. El **managed device** es una pieza de la red que reside en la **managed network** (un host, un router, un middlebox, etc).
3. La **Management Information Base**, que es la BDD de información de la red, que puede ser tan compleja como una BDD SQL o tan simple como un contador de paquetes UDP recibidos.
4. El **network management agent** es el proceso que corre en el managed device y se comunica con el managing server.
5. El **network management protocol** es el protocolo que corre entre el managing server y los managed devices.

SNMP es un ejemplo, es Simple Network Management Protocol.